

实验三：分治法

班级: 7 班

姓名: 卢瑶

学号: 20211018341

● 实验内容

第 1 关：利用分治法求一组数据中最大的两个数和最小的两个数。

```
#include <iostream>
#include <algorithm>

using namespace std;
#define max(x,y) ((x)>(y)?(x):(y)) //don't need add ';'
#define min(x,y) ((x)<(y)?(x):(y))

void MaxMin(int a[], int low, int high, int &maxe, int &maxel, int &mine, int &minel){ //
    //首先是递归出口(问题规模缩小到 1 or NULL)
    if(low == high){ //当只有一个元素时
        maxe = mine = a[low];
        maxel = minel = a[low];
    }
    else if(low == high - 1){ //只有两个元素时, 只有 1 个最大最小
        maxe = max(a[low], a[high]);
        maxel = min(a[low], a[high]);
        mine = min(a[low], a[high]);
        minel = max(a[low], a[high]);
    }
    else{
        int mid = (low + high) / 2;
        cout << mid << endl;
        int lmaxe, lmine, lmaxel, lminel;
        MaxMin(a, low, mid, lmaxe, lmaxel, lmine, lminel);
        int rmaxe, rmine, rmaxel, rminel;
        MaxMin(a, mid + 1, high, rmaxe, rmaxel, rmine, rminel);
        //回溯的时候 要进行比较!! (递归回溯)

        //求数组最大的两个数
        maxe = max(lmaxe, rmaxe);
        if(min(lmaxe, rmaxe) > max(lmaxel, rmaxel)) //两个最大在同一边
            maxel = min(lmaxe, rmaxe);
        else //不在同一边
            maxel = max(lmaxel, rmaxel); //catious about detail !

        //求数组最小的两个数
        mine = min(lmine, rmine);
        if(max(lmine, rmine) < min(lminel, rminel)) //注意多参数的利用!
            minel = max(lmine, rmine);
        else
            minel = min(lminel, rminel);
    }
    cout << "first:" << maxe << "âââ " << mine << endl;
    cout << maxel << " " << minel << endl;
}

int main(int argc, char *argv[]) {
```

```

int a[] = {5, 4, 2, 8, 1, 7}; //0 - 5
int min1, max1, min2, max2;
MaxMin(a, 0, 5, max1, max2, min1, min2);

cout << "Max1:" << max1 << " " << "Min1:" << min1<<endl;
cout << "Max2:" << max2 << " " << "Min2:" << min2<<endl;
}

```

复杂度分析:

$O(n)$

第2关：利用分治法求一组数据的和。

```

int Sum(int a[], int low, int high){
    //二叉遍历 数组即可~
    int mid;
    int sum = 0;
    if(low == high){ //分治到只有一个元素
        return a[low];
    }
    else if(low < high){
        mid = (low + high) / 2;
        return Sum(a, low, mid) + Sum(a, mid + 1, high);
    }
    return 0;
}

```

复杂度分析:

$O(n)$

第3关：对于给定的 n 个元素的数组 $a[0:n-1]$ ，要求从中找出第 k 小的元素。

```

#include <stdio.h>
#include <math.h>

int partition(int a[], int i, int j){
    int tmp = a[i];
    while(i != j){
        while(i < j && tmp <= a[j])
            j--;
        a[i] = a[j];
        while(i < j && tmp >= a[i])
            i++;
        a[j] = a[i];
    }
    a[i] = tmp; //当 i = j
    return i;
}

void QuickSort(int a[], int i, int j){
    if(i < j){
        int tmp = partition(a, i, j);
        QuickSort(a, i, tmp - 1);
        QuickSort(a, tmp + 1, j);
    }
}

```

```

/*
测试用例
10
1 3 5 7 9 10 8 6 4 2
*/
int main(int argc, char *argv[]) {
    int n, m;
    scanf("%d %d", &n, &m);
    int a[n];
    for(int i = 0; i < n; i++) scanf("%d", &a[i]);

    QuickSort(a, 0, n-1); //别忘了-1

    printf("第%d 小的元素是%d", m, a[m-1]);
}

```

复杂度分析 (利用快速排序):

$O(n \log_2 n)$

● 实验目的

掌握分治法设计思想；理解解决实际问题的关键要素——划分和合并，建立应用分治法解决实际问题的能力；了解递归与分治的关系。

● 实验总结

1. 了解了递归与分治的关系，对于大问题来说，递归就是为了“降维”，将大问题化为小问题，直到最简单的问题，然后在向上回溯解决问题，到了顶层问题也就结束了，但过程千万注意条件的判断。
2. 深刻体会到了划分和合并的过程。
3. 对于大问题，且每个小问题都有相同的性质的，一定要巧妙利用分治法解决问题！